Lecture L07

*Blind review — please score this lecture on row **L07** in the Excel form (1–10 per dimension).*

Introduction to Normalization Forms and Database Schema Design

Imagine a scenario where a single mistake in a database can lead to incorrect orders, misplaced shipments, and ultimately, dissatisfied customers. This is precisely what happens when a database is not properly normalized. By the end of this lecture, we will be able to define and explain the concepts of 1NF, 2NF, 3NF, and BCNF, and their roles in database design. We will also identify and apply the rules for normalizing relations to minimize data redundancy and anomalies, analyze the trade-offs between normalization and performance in database design, design a database schema using normalization principles to ensure data consistency and scalability, and recognize common pitfalls and limitations in normalization and propose solutions.

Normalization is a crucial process in database design that aims to minimize data redundancy and anomalies. To understand why normalization is essential, let's consider a simple example. Suppose we have a relation called "Orders" with attributes "Order ID", "Customer Name", "Order Date", and "Product Names". If we store multiple product names in a single cell, we violate the principle of atomicity, which can lead to data inconsistencies. This is where normalization comes in – to ensure that each cell in the relation contains a single value. Think of normalization as a way to organize your data into a logical and consistent structure, much like a librarian organizes books on a shelf. In fact, this concept is also crucial in data warehousing, where a well-normalized database can help ensure that data is properly aggregated and analyzed, leading to more accurate business insights. For instance, in a retail company, a normalized database can help track sales trends and customer behavior, enabling data-driven decisions.

Having defined the importance of normalization, we now delve into the definitions of keys and normal forms. A key is a set of attributes that uniquely identifies a tuple in a relation. The normal form of a relation refers to the highest normal form condition that it meets, indicating the degree to which it has been normalized. A key plays a crucial role in defining the normal forms of a relation. To illustrate this concept, consider a social security number as a unique identifier for an individual. Just as a social security number uniquely identifies a person, a key uniquely identifies a tuple in a relation. This concept is also similar to the idea of data fingerprinting in cybersecurity, where a unique identifier is used to track and verify data integrity.

Definitions of Keys and Normal Forms

To develop an intuition about keys and normal forms, think of a key as a unique identifier for each row in your relation, and normal forms as a way to ensure that your data is organized in a logical and consistent manner. Before diving into the formal definitions, it is essential to understand that keys and normal forms are fundamental concepts in database design. Try to think of a real-world scenario where you have to identify a specific item from a collection. How would you do it? You would likely use a unique identifier, such as a name or a number. This is similar to how keys work in database design. In fact, this concept is also crucial in machine learning pipelines, where data preprocessing and feature engineering rely heavily on the quality of the data, which in turn depends on the normalization of the database. For instance, in a customer segmentation task, a well-normalized database can help ensure that each customer's information is accurately represented, leading to more effective clustering and targeting. Additionally, in a recommendation system, a normalized database can help improve the accuracy of recommendations by reducing data redundancy and anomalies.

First Normal Form (1NF)

A relation is in 1NF if each cell in the relation contains a single value. This is the most basic normal form, and it is essential to ensure that each attribute has a unique value. To illustrate this concept, consider the "Orders" relation from our previous example. If we have a cell that contains multiple product names, we are not in 1NF. However, if we create a separate row for each product, we achieve 1NF. For instance, suppose we have a relation "Students" with attributes "Student ID", "Name", and "Courses". If we store multiple courses in a single cell, we are not in 1NF. To normalize this relation, we can create a separate row for each course, resulting in a relation that meets the 1NF condition. As an exercise, try to think of a scenario where you have to store multiple values in a single cell. How would you normalize the data? In machine learning, 1NF is crucial when preparing data for training models. For example, when building a recommender system, ensuring that each user's ratings are stored in a separate row can help improve the accuracy of the recommendations. Additionally, in a natural language processing task, ensuring that each text sample is associated with the correct label and metadata can help improve the accuracy of the model.

To further illustrate the importance of 1NF in machine learning, let's consider a concrete example. Suppose we are building a sentiment analysis model, and we have a dataset of text reviews with corresponding sentiment labels. If we store multiple reviews in a single cell, we would not be in 1NF. However, if we create a separate row for each review, we can ensure that each review is associated with the correct sentiment label, leading to more accurate model training. For instance, if we have a dataset with the following structure:

Review ID	Text Review	Sentiment Label
1	"I love this product"	Positive
1	"I hate this product"	Negative
2	"This product is okay"	Neutral

We can normalize this dataset by creating a separate row for each review, resulting in the following structure:

Review ID	Text Review	Sentiment Label
1	"I love this product"	Positive
1	"I hate this product"	Negative
2	"This product is okay"	Neutral

This normalized dataset can then be used to train a sentiment analysis model, leading to more accurate predictions.

Second Normal Form (2NF)

A relation is in 2NF if it is in 1NF and each non-key attribute depends on the entire primary key. This means that if a relation has a composite key, each non-key attribute should depend on all the attributes in the key. Let's consider an example to develop our intuition. Suppose we have a relation "Orders" with a composite key "Order ID" and "Product ID", and a non-key attribute "Order Date". If the "Order Date" depends only on the "Order ID" and not on the "Product ID", we are not in 2NF. To achieve 2NF, we need to ensure that each non-key attribute depends on the entire primary key. For example, suppose we have a relation "Sales" with attributes "Order ID", "Product ID", "Salesperson ID", and "Commission". If the commission depends only on the salesperson ID and not on the order ID and product ID, we are

not in 2NF. To normalize this relation, we can create a separate relation "Salespersons" with attributes "Salesperson ID" and "Commission", resulting in a relation that meets the 2NF condition. Think of 2NF as a way to ensure that each non-key attribute is fully dependent on the primary key, much like a student's grade depends on their performance in all subjects. This concept is also similar to the idea of feature engineering in machine learning, where each feature should be relevant to the target variable. For instance, in a predictive maintenance task, ensuring that each sensor reading is associated with the correct machine and timestamp can help improve the accuracy of the predictions.

To further illustrate the importance of 2NF in machine learning, let's consider a concrete example. Suppose we are building a predictive maintenance model, and we have a dataset of sensor readings with corresponding machine IDs and timestamps. If we store multiple sensor readings in a single cell, we would not be in 1NF. However, if we create a separate row for each sensor reading, we can ensure that each sensor reading is associated with the correct machine ID and timestamp, leading to more accurate model training. For instance, if we have a dataset with the following structure:

Machine ID	Sensor Reading	Timestamp
1	10, 20, 30	2022-01-01
2	40, 50, 60	2022-01-02

We can normalize this dataset by creating a separate row for each sensor reading, resulting in the following structure:

Machine ID	Sensor Reading	Timestamp
1	10	2022-01-01
1	20	2022-01-01
1	30	2022-01-01
2	40	2022-01-02
2	50	2022-01-02
2	60	2022-01-02

This normalized dataset can then be used to train a predictive maintenance model, leading to more accurate predictions.

Third Normal Form (3NF)

A relation is in 3NF if it is in 2NF and there are no transitive dependencies. A transitive dependency occurs when a non-key attribute depends on another non-key attribute. Suppose we have a relation "Employees" with attributes "Employee ID", "Name", and "Department Name". If the department name depends on the employee's name, we have a transitive dependency. To achieve 3NF, we need to eliminate transitive dependencies by creating separate relations for each non-key attribute. For example, suppose we have a relation "Courses" with attributes "Course ID", "Course Name", and "Instructor Name". If the instructor name depends on the course name, we are not in 3NF. To normalize this relation, we can create a separate relation "Instructors" with attributes "Instructor ID" and "Instructor Name", and another relation "Course Instructors" with attributes "Course ID" and "Instructor ID", resulting in relations that meet the 3NF condition. As an exercise, try to identify a transitive dependency in a real-world scenario and think of how you would eliminate it. This is similar to the concept of data preprocessing in machine learning, where we need to handle missing values and outliers to ensure that our data is consistent and accurate. For instance, in a natural language processing task, ensuring that each text sample is associated with the correct label and metadata can help improve the accuracy of the model.

To further illustrate the importance of 3NF in machine learning, let's consider a concrete example. Suppose we are building a text classification model, and we have a dataset of text samples with corresponding labels and metadata. If we store multiple labels in a single cell, we would not be in 1NF. However, if we create a separate row for each label, we can ensure that each label is associated with the correct text sample and metadata, leading to more accurate model training. For instance, if we have a dataset with the following structure:

Text Sample	Label	Metadata
"This is a positive review"	Positive, Negative	Review ID 1
"This is a negative review"	Positive, Negative	Review ID 2

We can normalize this dataset by creating a separate row for each label, resulting in the following structure:

Text Sample	Label	Metadata
"This is a positive review"	Positive	Review ID 1
"This is a positive review"	Negative	Review ID 1
"This is a negative review"	Positive	Review ID 2
"This is a negative review"	Negative	Review ID 2

However, this is still not in 3NF, as the label depends on the text sample, which is a non-key attribute. To achieve 3NF, we can create a separate relation "Labels" with attributes "Label ID" and "Label Name", and another relation "Text Sample Labels" with attributes "Text Sample ID" and "Label ID", resulting in relations that meet the 3NF condition.

Text Sample ID	Text Sample	Metadata
1	"This is a positive review"	Review ID 1
2	"This is a negative review"	Review ID 2

Label ID	Label Name
1	Positive
2	Negative

Text Sample ID	Label ID
1	1
1	2
2	1
2	2

This normalized dataset can then be used to train a text classification model, leading to more accurate predictions.

Boyce-Codd Normal Form (BCNF)

A relation is in BCNF if it is in 3NF and there are no transitive dependencies, and each determinant is a candidate key. A determinant is an attribute that determines another attribute. Suppose we have a relation "Students" with attributes "Student ID", "Name", and "Course ID". If the course ID depends on the student ID, we have a determinant. To achieve BCNF, we need to ensure that each determinant is a candidate key. For example, suppose we have a relation "Orders" with attributes "Order ID", "Customer ID", and "Product ID". If the product ID depends on the order ID, we are not in BCNF. To normalize this relation, we can create a separate relation "Order Products" with attributes "Order ID" and "Product ID", resulting in a relation that meets the BCNF condition. As an exercise, try to identify a determinant in a real-world scenario and think of how you would ensure that it is a candidate key. This is similar to the concept of feature selection in machine learning, where we need to select the most relevant features to improve model accuracy. For instance, in a customer churn prediction task, ensuring that each feature is relevant to the target variable can help improve the accuracy of the predictions.

To further illustrate the importance of BCNF in machine learning, let's consider a concrete example. Suppose we are building a customer churn prediction model, and we have a dataset of customer information with corresponding churn labels. If we store multiple features in a single cell, we would not be in 1NF. However, if we create a separate row for each feature, we can ensure that each feature is associated with the correct customer information and churn label, leading to more accurate model training. For instance, if we have a dataset with the following structure:

Customer ID	Feature 1, Feature 2, Feature 3	Churn Label
1	10, 20, 30	0
2	40, 50, 60	1

We can normalize this dataset by creating a separate row for each feature, resulting in the following structure:

Customer ID	Feature 1	Feature 2	Feature 3	Churn Label
1	10	20	30	0
2	40	50	60	1

However, this is still not in BCNF, as the feature 1 depends on the customer ID, which is a determinant. To achieve BCNF, we can create a separate relation "Features" with attributes "Feature ID" and "Feature Name", and another relation "Customer Features" with attributes "Customer ID" and "Feature ID", resulting in relations that meet the BCNF condition.

Customer ID	Churn Label
1	0
2	1

Feature ID	Feature Name
1	Feature 1
2	Feature 2
3	Feature 3

Customer ID	Feature ID	Feature Value
1	1	10
1	2	20
1	3	30
2	1	40
2	2	50
2	3	60

This normalized dataset can then be used to train a customer churn prediction model, leading to more accurate predictions.

In conclusion, normalization is a crucial process in database design that aims to minimize data redundancy and anomalies. By understanding the concepts of 1NF, 2NF, 3NF, and BCNF, we can design a database schema that ensures data consistency and scalability. Additionally, by applying these concepts to machine learning tasks, we can improve the accuracy of our models and make more informed decisions. As an exercise, try to think of a real-world scenario where you have to design a database schema and apply normalization principles to ensure data consistency and scalability. How would you do it? What are the benefits and limitations of normalization in this context?